

TRAVELOOP

Full Stack Tech Stack, Architecture & Build Guide

8-Hour Hackathon Edition • Animesh & Kajol

1. Tech Stack Recommendation

1.1 Frontend

- React 18 + Vite (not Create React App — 10x faster builds)
- TailwindCSS + Headless UI (for accessible components)
- React Query (TanStack Query) — superior data fetching/caching
- Zustand — lightweight state management (simpler than Redux)
- React Router v6 — navigation
- Framer Motion — smooth animations
- Recharts — budget visualizations

Why: Vite gives instant HMR, TailwindCSS prevents CSS bloat, React Query handles caching/optimistic updates (looks real-time without WebSockets initially).

1.2 Backend

- Spring Boot 3.2 (Java 17+)
- PostgreSQL 15 — better JSON support than MySQL for activities/metadata
- Spring Data JPA + Hibernate
- Spring Security + JWT
- Spring Validation
- ModelMapper — DTO conversions
- Lombok — reduce boilerplate
- Swagger / OpenAPI — API documentation

1.3 AI Integration (The Differentiator)

- OpenAI API or Anthropic Claude API for:
 - Smart itinerary generation from natural language
 - Budget optimization suggestions
 - Activity recommendations based on preferences
- Implement as a separate service layer — don't couple tightly

2. Design Philosophy — Standing Out

2.1 Color Palette (NOT Generic AI)

Variable	Hex	Usage
--primary	#2D5F5D	Deep Teal — trust, journey
--secondary	#E8956F	Terracotta — warmth, adventure
--accent	#F4A261	Sunset Orange — energy
--neutral-dark	#2C3639	Charcoal
--neutral-light	#F5F1E8	Warm White
--success	#52796F	Sage Green
--background	#FEFEFE	Clean White

2.2 Design Principles

- Cards with subtle shadows (not glassmorphism)
- Generous whitespace (Airbnb-style breathing room)
- Rounded corners (8px standard, 16px for cards)
- Micro-interactions (hover states, loading skeletons)
- Typography hierarchy — Inter or Manrope font

2.3 Skip Figma — Use This Approach

- Hour 0–0.5: Quick Excalidraw wireframes (just boxes and flow)
- Build with component library base: Use shadcn/ui (Tailwind-based, copy-paste, customizable)
- Design tokens first: Define spacing scale, colors, typography in tailwind.config.js

3. Database Schema (Critical for Judging)

3.1 Core Tables Overview

Table	Primary Key	Description
users	user_id (UUID)	Authentication and profile data
trips	trip_id (UUID)	Trip metadata, owner, dates, sharing
cities	city_id (UUID)	City data with coordinates and cost index
trip_stops	stop_id (UUID)	Ordered stops within a trip
activities	activity_id (UUID)	Activity catalogue per city

trip_activities	trip_activity_id (UUID)	Scheduled activities per stop
budgets	budget_id (UUID)	Estimated vs actual per category
packing_items	item_id (UUID)	Packing checklist per trip
trip_notes	note_id (UUID)	Journal notes per trip/stop

3.2 SQL DDL — Users & Trips

```
CREATE TABLE users (
  user_id          UUID PRIMARY KEY,
  email            VARCHAR(255) UNIQUE NOT NULL,
  password_hash    VARCHAR(255) NOT NULL,
  full_name        VARCHAR(100),
  profile_photo_url VARCHAR(500),
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE trips (
  trip_id          UUID PRIMARY KEY,
  user_id          UUID REFERENCES users(user_id) ON DELETE CASCADE,
  trip_name        VARCHAR(200) NOT NULL,
  description       TEXT,
  start_date       DATE NOT NULL,
  end_date         DATE NOT NULL,
  cover_photo_url  VARCHAR(500),
  is_public        BOOLEAN DEFAULT FALSE,
  share_token      VARCHAR(100) UNIQUE,
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3.3 SQL DDL — Cities, Stops & Activities

```
CREATE TABLE cities (
  city_id          UUID PRIMARY KEY,
  city_name        VARCHAR(100) NOT NULL,
  country          VARCHAR(100) NOT NULL,
  latitude         DECIMAL(10, 8),
  longitude        DECIMAL(11, 8),
  cost_index       INTEGER,           -- 1-5 scale
  popularity_score INTEGER,
  metadata         JSONB             -- flexible additional info
);

CREATE TABLE trip_stops (
  stop_id          UUID PRIMARY KEY,
  trip_id          UUID REFERENCES trips(trip_id) ON DELETE CASCADE,
  city_id          UUID REFERENCES cities(city_id),
  arrival_date     DATE NOT NULL,
  departure_date   DATE NOT NULL,
  stop_order       INTEGER NOT NULL,
  notes            TEXT,
  UNIQUE(trip_id, stop_order)
);

CREATE TABLE activities (
```

```
    activity_id      UUID PRIMARY KEY,
    city_id         UUID REFERENCES cities(city_id),
    activity_name    VARCHAR(200) NOT NULL,
    description      TEXT,
    category         VARCHAR(50),      -- sightseeing, food, adventure
    estimated_cost   DECIMAL(10, 2),
    duration_minutes INTEGER,
    image_url        VARCHAR(500)
);
```

3.4 SQL DDL — Trip Activities, Budgets, Packing & Notes

```
CREATE TABLE trip_activities (
    trip_activity_id UUID PRIMARY KEY,
    stop_id          UUID REFERENCES trip_stops(stop_id) ON DELETE CASCADE,
    activity_id       UUID REFERENCES activities(activity_id),
    scheduled_date    DATE,
    scheduled_time    TIME,
    actual_cost       DECIMAL(10, 2),
    status            VARCHAR(20) DEFAULT 'planned' -- planned, completed, skipped
);

CREATE TABLE budgets (
    budget_id        UUID PRIMARY KEY,
    trip_id          UUID REFERENCES trips(trip_id) ON DELETE CASCADE,
    category          VARCHAR(50),      -- transport, accommodation, activities, meals
    estimated_amount  DECIMAL(10, 2),
    actual_amount     DECIMAL(10, 2) DEFAULT 0
);

CREATE TABLE packing_items (
    item_id          UUID PRIMARY KEY,
    trip_id          UUID REFERENCES trips(trip_id) ON DELETE CASCADE,
    item_name        VARCHAR(100) NOT NULL,
    category          VARCHAR(50),      -- clothing, documents, electronics
    is_packed        BOOLEAN DEFAULT FALSE,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE trip_notes (
    note_id          UUID PRIMARY KEY,
    trip_id          UUID REFERENCES trips(trip_id) ON DELETE CASCADE,
    stop_id          UUID REFERENCES trip_stops(stop_id) ON DELETE CASCADE,
    note_content      TEXT NOT NULL,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for performance
CREATE INDEX idx_trips_user      ON trips(user_id);
CREATE INDEX idx_trip_stops_trip ON trip_stops(trip_id);
CREATE INDEX idx_activities_city ON activities(city_id);
CREATE INDEX idx_budgets_trip   ON budgets(trip_id);
```

4. Backend Architecture (SOLID + Patterns)

4.1 Project Structure

```
src/main/java/com/traveloop/
├── config/
│   ├── SecurityConfig.java
│   ├── CorsConfig.java
│   └── OpenAPIConfig.java
├── model/
│   ├── entity/           (User, Trip, City, TripStop, Activity ...)
│   └── dto/
│       ├── request/      (CreateTripRequest, AddStopRequest ...)
│       └── response/      (TripResponse, ItineraryResponse ...)
├── repository/           (UserRepository, TripRepository ...)
├── service/
│   ├── interfaces/       (TripService, AIRecommendationService ...)
│   └── impl/              (TripServiceImpl, AIRecommendationServiceImpl ...)
├── controller/           (AuthController, TripController ...)
├── exception/             (GlobalExceptionHandler, ResourceNotFoundException ...)
└── util/                  (JwtUtil, ValidationUtil)
```

4.2 Key Design Patterns

- Repository Pattern — provided by Spring Data JPA
- Service Layer Pattern — business logic separate from controllers
- DTO Pattern — never expose entities directly
- Builder Pattern — for complex object creation (Trip with stops)
- Strategy Pattern — for AI recommendation algorithms

4.3 Example Service (SOLID Compliance)

```
@Service
@Transactional
public class TripServiceImpl implements TripService {

    private final TripRepository tripRepository;
    private final TripStopRepository tripStopRepository;
    private final BudgetCalculationService budgetService;
    private final ModelMapper modelMapper;

    // Constructor injection (D – Dependency Inversion)
    public TripServiceImpl(TripRepository tripRepository,
                          TripStopRepository tripStopRepository,
                          BudgetCalculationService budgetService,
                          ModelMapper modelMapper) {
        this.tripRepository = tripRepository;
        this.tripStopRepository = tripStopRepository;
        this.budgetService = budgetService;
        this.modelMapper = modelMapper;
    }

    @Override
    public TripResponse createTrip(CreateTripRequest request, UUID userId) {
        // S – Single Responsibility: Only handles trip creation
        validateTripDates(request);

        Trip trip = Trip.builder()
            .userId(userId)
            .tripName(request.getTripName())
```

```

        .description(request.getDescription())
        .startDate(request.getStartDate())
        .endDate(request.getEndDate())
        .shareToken(generateShareToken())
        .build();

        Trip savedTrip = tripRepository.save(trip);
        budgetService.initializeDefaultBudgets(savedTrip.getTripId());
        return modelMapper.map(savedTrip, TripResponse.class);
    }

    // O - Open/Closed: Extensible without modification
    private void validateTripDates(CreateTripRequest request) {
        if (request.getStartDate().isAfter(request.getEndDate())) {
            throw new InvalidTripDataException("Start date must be before end
date");
        }
    }
}

```

4.4 Request Validation (Backend)

```

@Data @Builder
public class CreateTripRequest {

    @NotBlank(message = "Trip name is required")
    @Size(min = 3, max = 200)
    private String tripName;

    @NotNull(message = "Start date is required")
    @Future(message = "Start date must be in the future")
    private LocalDate startDate;

    @NotNull(message = "End date is required")
    private LocalDate endDate;

    @Size(max = 1000)
    private String description;

    @AssertTrue(message = "End date must be after start date")
    private boolean isValidDateRange() {
        return endDate == null || startDate == null ||
            endDate.isAfter(startDate);
    }
}

```

4.5 Pagination & Performance

```

@GetMapping("/trips")
public Page<TripResponse> getTrips(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    Principal principal) {

    Pageable pageable = PageRequest.of(page, size,
        Sort.by("createdAt").descending());
    return tripService.getUserTrips(principal.getName(), pageable);
}

```

```
}
```

5. AI Integration Strategy (The Differentiator)

5.1 Feature 1 — Smart Itinerary Generator

```
@Service
public class AIRecommendationServiceImpl implements AIRecommendationService {

    private final RestTemplate restTemplate;

    @Value("${anthropic.api.key}")
    private String apiKey;

    public List<ActivitySuggestion> generateSmartItinerary(TripContext context) {
        String prompt = buildPrompt(context);

        ClaudeRequest request = ClaudeRequest.builder()
            .model("claude-sonnet-4-20250514")
            .maxTokens(1000)
            .messages(List.of(new Message("user", prompt)))
            .build();

        ClaudeResponse response = callClaudeAPI(request);
        return parseActivitySuggestions(response);
    }

    private String buildPrompt(TripContext context) {
        return String.format("""
            Generate a day-by-day itinerary for a trip:
            - Cities: %s
            - Duration: %d days
            - Budget: $%s
            - Interests: %s
            Return JSON array of activities with: name, category,
            estimated_cost, duration_minutes, description
            """,
            context.getCities(), context.getDurationDays(),
            context.getBudget(), context.getInterests());
    }
}
```

5.2 Feature 2 — Budget Optimization Assistant

- Analyze user's budget allocation
- Suggest reallocation based on destination cost indices
- Warn about over-budget days

6. API Endpoints (RESTful Design)

6.1 Authentication

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
GET  /api/auth/me
```

6.2 Trips

```
GET /api/trips
POST /api/trips
GET  /api/trips/{id}
PUT  /api/trips/{id}
DELETE /api/trips/{id}
GET  /api/trips/{id}/share-link
GET  /api/trips/shared/{token}
```

6.3 Itinerary

```
POST /api/trips/{tripId}/stops
PUT  /api/trips/{tripId}/stops/{stopId}
DELETE /api/trips/{tripId}/stops/{stopId}
POST /api/trips/{tripId}/stops/{stopId}/activities
GET  /api/trips/{tripId}/itinerary
```

6.4 Search

```
GET /api/cities/search?q={query}
GET /api/activities/search?cityId={id}&category={cat}
```

6.5 Budget

```
GET /api/trips/{tripId}/budget
PUT /api/trips/{tripId}/budget/{category}
```

6.6 AI Features

```
POST /api/ai/generate-itinerary
POST /api/ai/optimize-budget
POST /api/ai/recommend-activities
```

6.7 Packing & Notes

```
GET /api/trips/{tripId}/packing-list
POST /api/trips/{tripId}/packing-list
PUT  /api/trips/{tripId}/packing-list/{itemId}

GET /api/trips/{tripId}/notes
POST /api/trips/{tripId}/notes
PUT  /api/notes/{noteId}
DELETE /api/notes/{noteId}
```

7. Frontend Architecture

7.1 Project Structure

```
src/
├── components/
│   └── common/      (Button, Card, Input, Modal)
```


— layout/	(Navbar, Sidebar, Footer)
— trip/	(TripCard, TripList, CreateTripForm)
— itinerary/	(ItineraryBuilder, StopCard, ActivitySelector, TimelineView)
— budget/	(BudgetOverview, CategoryBreakdown, CostChart)
— pages/	(Login, Dashboard, TripDetail, ItineraryBuilder)
— hooks/	(useTrips, useItinerary, useAuth)
— services/	(api.js, tripService.js, authService.js)
— store/	(authStore.js, tripStore.js)
— utils/	(constants.js, formatters.js)

7.2 React Query Setup (Performance Boost)

```
// services/tripService.js
export const tripService = {
  getTrips: async () => (await api.get('/api/trips')).data,
  createTrip: async (tripData) => (await api.post('/api/trips', tripData)).data,
  getItinerary: async (tripId) => (await
api.get(`/api/trips/${tripId}/itinerary`)).data,
};

// hooks/useTrips.js
export function useTrips() {
  return useQuery({
    queryKey: ['trips'],
    queryFn: tripService.getTrips,
    staleTime: 5 * 60 * 1000, // 5 minutes
  });
}

export function useCreateTrip() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: tripService.createTrip,
    onSuccess: () => queryClient.invalidateQueries(['trips']),
  });
}
```

7.3 Form Validation (Frontend — Zod)

```
const tripSchema = z.object({
  tripName: z.string().min(3).max(200),
  startDate: z.date().min(new Date()),
  endDate: z.date(),
  description: z.string().max(1000).optional()
}).refine(
  data => data.endDate > data.startDate,
  { message: 'End date must be after start date' }
);

function CreateTripForm() {
  const { register, handleSubmit, formState: { errors } } =
    useForm({ resolver: zodResolver(tripSchema) });
  // Form implementation ...
}
```

8. Performance Optimizations

8.1 Backend

- Hibernate N+1 Prevention — use @EntityGraph or JOIN FETCH
- Pagination — all list endpoints return paginated results
- Caching — @Cacheable on city/activity search
- Connection Pooling — HikariCP configuration
- Async Processing — for AI API calls

8.2 Frontend

- Code Splitting — React.lazy() for routes
- Image Optimization — WebP format, lazy loading
- Bundle Analysis — keep bundle < 200 KB initial
- Debouncing — search inputs with 300 ms delay
- Optimistic Updates — via React Query mutations

9. Work Division Strategy

9.1 Phase 1 — Foundation (Hour 0–2)

TOGETHER (Pair programming)

- Database schema design (30 min)
- API contract definition (30 min)
- Project scaffolding (30 min)
- Git workflow setup (30 min)

Animesh (Backend)	Kajol (Frontend)
Spring Boot project setup	React + Vite project setup
Security configuration (JWT)	Tailwind configuration + design tokens
User authentication endpoints	Component library setup (shadcn/ui)
Database initialization scripts	Auth pages (Login/Signup)

9.2 Phase 2 — Core Features (Hour 2–5)

Animesh (Backend)	Kajol (Frontend)
Trip CRUD APIs	Dashboard page
Itinerary builder backend	Trip creation flow

City & Activity search APIs	Itinerary builder UI
Budget calculation service	City/Activity search interface
AI integration (Claude API)	Budget visualization components

9.3 Phase 3 — Advanced Features (Hour 5–7)

Animesh (Backend)	Kajol (Frontend)
Public trip sharing (share tokens)	Itinerary view/timeline
Packing checklist backend	Packing checklist UI
Trip notes/journal backend	Trip sharing UI
Performance optimization (caching)	Loading states & error handling / Responsive polish

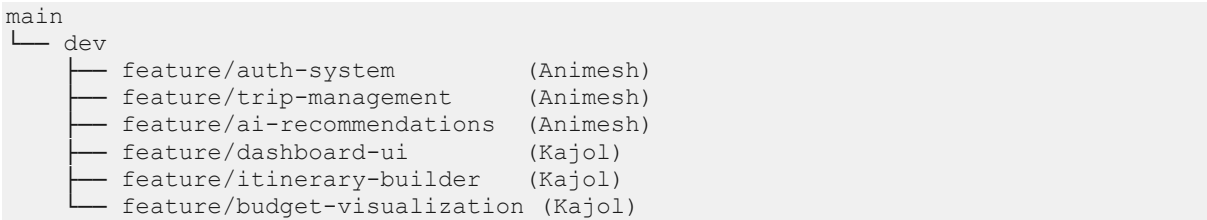
9.4 Phase 4 — Polish & Deploy (Hour 7–8)

TOGETHER

- Integration testing
- Bug fixes
- Deployment — Vercel frontend + Railway/Render backend
- Presentation deck preparation

10. Git Strategy (Show Genuine Collaboration)

10.1 Branch Structure



10.2 Commit Strategy

- Meaningful commits: e.g. "feat: Add JWT authentication with refresh tokens"
- Both commit frequently (every 20–30 minutes)
- Cross-review: Kajol reviews Animesh's backend PRs, and vice versa
- Pair commits: use co-authored commits for integrated features

Co-authored-by: Kajol <kajol@email.com>

11. Unique Features (Stand Out from Others)

Feature 1 — AI-Powered Smart Planner

- Natural language input: "Plan a 5-day trip to Southeast Asia under \$2000"
- Claude API generates structured itinerary
- Users can accept / modify / regenerate

Feature 2 — Real-Time Budget Predictor

- As users add activities, live budget calculation updates
- Visual warnings for over-budget days
- ML-based cost prediction using city cost indices

Feature 3 — Collaborative Trip Planning

- Share trip link with friends
- Real-time cursor positions (if time permits with WebSockets)
- Comment system on activities

Feature 4 — Interactive Timeline View

- Horizontal scrollable timeline
- Drag-and-drop activities
- Visual day separators

Feature 5 — Smart Packing Assistant

- Auto-suggest items based on destination weather, trip duration, and planned activities

12. Deployment Strategy

Layer	Platform	Notes
Frontend	Vercel	Instant deploy from GitHub; env vars for API URL
Backend	Railway or Render	Free tier with PostgreSQL; env vars for API keys; CORS configured
Database	Neon or Railway PostgreSQL	Serverless Postgres option available via Neon

13. 8-Hour Timeline Breakdown

Time	Animesh	Kajol
0:00 – 0:30	TOGETHER: Schema design, API contracts	TOGETHER: Schema design, API contracts
0:30 – 1:00	Project setup + Security	Project setup + Design system
1:00 – 2:00	User auth + JWT	Login / Signup UI
2:00 – 3:00	Trip CRUD + validation	Dashboard + Trip creation
3:00 – 4:00	Itinerary endpoints + search	Itinerary builder UI
4:00 – 5:00	AI integration (Claude API)	Budget visualization
5:00 – 6:00	Budget calculation service	Activity search UI
6:00 – 7:00	Sharing + packing list	Timeline view + sharing UI
7:00 – 7:30	TOGETHER: Integration testing	TOGETHER: Integration testing
7:30 – 8:00	TOGETHER: Deploy + presentation prep	TOGETHER: Deploy + presentation prep

14. Presentation Strategy

14.1 Demo Flow (5 minutes)

1. Login — Quick auth (pre-created account)
2. Dashboard — Show trip list
3. Create Trip — "Europe Summer 2024"
4. AI Magic — Natural language: "Plan 3 days in Paris with budget \$1500"
5. Itinerary Builder — Show AI-generated results
6. Customize — Add/remove activities, drag-and-drop
7. Budget View — Interactive charts
8. Share — Generate share link, open in incognito
9. Responsive — Show mobile view

14.2 Talking Points

Animesh (Backend focus):

- Database normalization (3NF)
- SOLID principles in service layer
- AI integration architecture
- Security implementation (JWT + password hashing)

Kajol (Frontend focus):

- Component reusability
- State management strategy
- Performance optimizations (React Query caching)
- Responsive design approach

BOTH mention:

- Git collaboration workflow
- API contract-first development
- Continuous integration testing

15. Final Checklist

15.1 Must-Have Features

- User authentication (JWT)
- Trip CRUD operations
- Multi-city itinerary builder
- City & activity search
- Budget tracking with visualization
- Public trip sharing
- AI-powered itinerary generation
- Responsive design (mobile + desktop)
- Proper validation (frontend + backend)
- Database relationships (foreign keys, indexes)

15.2 Nice-to-Have (If Time Permits)

- Packing checklist
- Trip notes / journal
- Email notifications
- Image uploads (Cloudinary)
- Advanced filters (date range, budget range)

15.3 Code Quality

- Follows SOLID principles
- No code duplication (DRY)
- Meaningful variable names
- Comments on complex logic only

- Error handling everywhere
- Input validation on all endpoints

15.4 Git Hygiene

- Meaningful commit messages
- Both team members committing regularly
- PR reviews with comments
- Feature branches, not direct commits to main

16. Differentiation Summary

You're building for production, not just for a demo. Think like you're launching to 10,000 users next month.

What	Why It Stands Out
AI Integration	Claude API for smart planning — most teams won't do this
Production-Grade Architecture	Not just CRUD — proper service layers, DI, patterns
Thoughtful Design	Earthy color palette, not generic neon gradients
Performance	React Query caching, pagination, optimistic updates
Real Collaboration	Genuine pair programming + cross-reviews in Git
Advanced DB Design	Proper normalization, indexes, foreign keys
Validation Everywhere	Both frontend (Zod) and backend (Spring Validation)
Deployment	Actually deployed, not just localhost